

# 软件系统数据模型设计说明书

---

## DjangoBlog 博客系统

| 项目     | 内容                   |
|--------|----------------------|
| 组号     | 17                   |
| 班级     | 软件工程 6 班             |
| 小组成员   | 何动、孙子杰、唐磊、唐文龙、吴政、程光文 |
| 本周文档整理 | 唐文龙                  |
| 项目名称   | DjangoBlog 博客系统      |
| 日期     | 2026 年 10 月 7 日      |

# 1 文档说明

## 1.1 编写目的

本文档用于说明 DjangoBlog 博客系统的数据库配置、Django Model 设计、模型之间的关系以及对应的数据库表结构。文档中的模型关系以项目 src 目录下的 models.py 和 settings.py 为依据，并通过 UML 类图进行可视化表达。

## 1.2 文档范围

本文档覆盖 accounts、blog、comments、oauth 和 servermanager 五个业务应用。Django 自带的认证、权限、会话和站点表在必要时进行说明，但不作为本系统的核心业务模型展开。

## 1.3 建模依据

- 数据库配置：src/djangoblog/settings.py 中的 DATABASES 和 AUTH\_USER\_MODEL。
- 模型定义：各业务应用 models.py 中的普通模型、外键、多对多字段、索引和约束。
- 架构分析：Django MTV 模式、URLConf、View、Model、Template 和 Middleware。
- 代码标注：模型、配置、路由和视图中的中文注释。

# 2 系统概述

DjangoBlog 是一个基于 Python 和 Django 的博客系统，后端使用 Django 5.x，默认数据库为 MySQL，字符集为 utf8mb4。系统具有文章发布、分类和标签管理、评论回复、OAuth 登录、站点配置、全文搜索和插件扩展等功能。

| 层次         | 项目位置              | 主要职责                |
|------------|-------------------|---------------------|
| Model      | 各 app 的 models.py | 定义数据结构、字段约束和实体关系    |
| Template   | src/templates     | 渲染页面、复用布局和组件        |
| View       | 各 app 的 views.py  | 处理请求、调用 Model 并返回响应 |
| URLConf    | 各 app 的 urls.py   | 将 URL 路径分发到对应 View  |
| Middleware | settings.py       | 处理安全、会话、认证和公共请求逻辑   |

## 3 数据库与模型分析

### 3.1 数据库配置

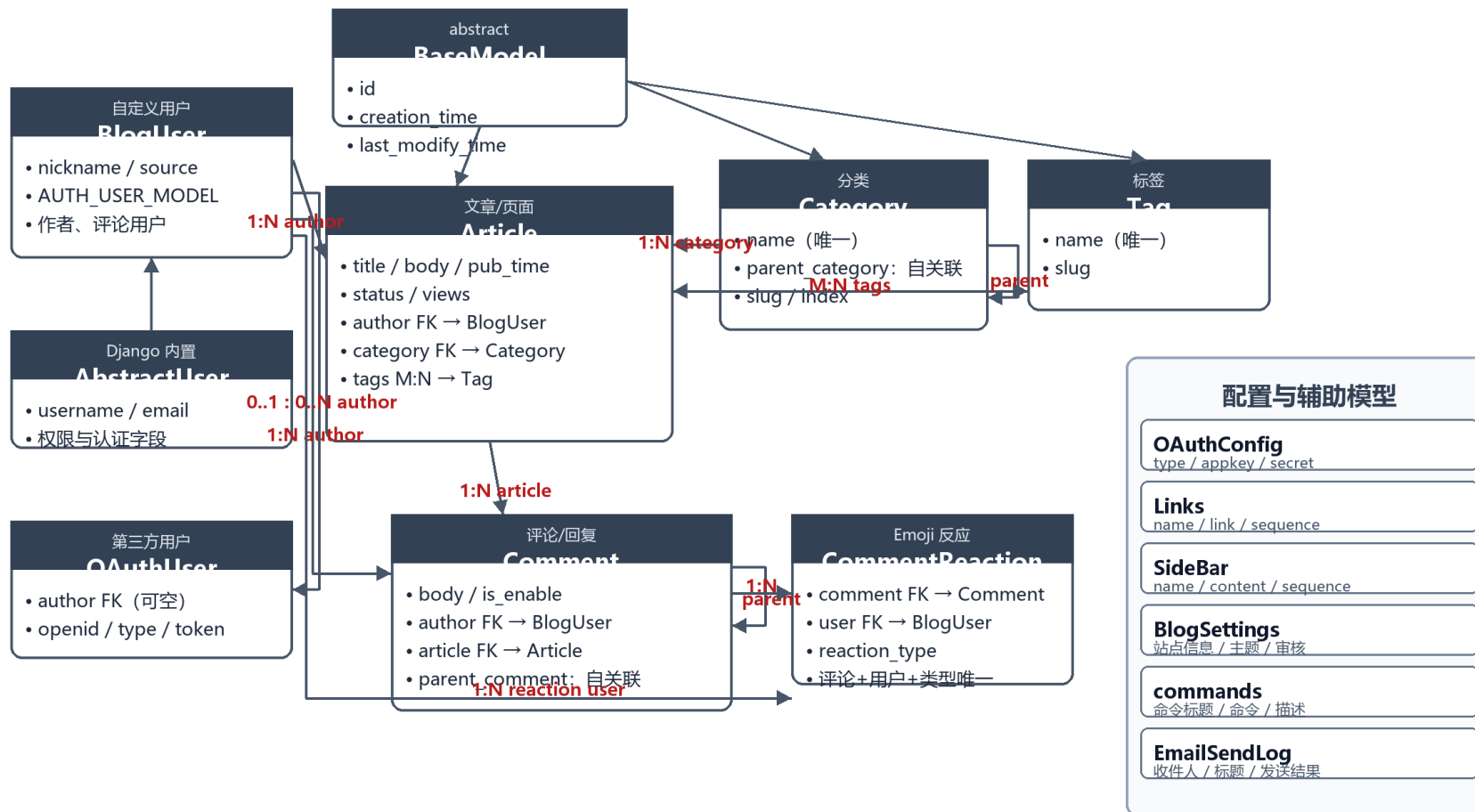
项目默认使用 MySQL 数据库，数据库名、用户名、密码、主机和端口都可以通过环境变量覆盖。代码中的默认值用于本地开发，部署时不应把生产密码提交到 Git 仓库。Django ORM 通过 migration 将模型变化同步为数据库表和字段。

### 3.2 模型角色划分

| 应用            | 模型              | 职责          | 主要关系                        |
|---------------|-----------------|-------------|-----------------------------|
| accounts      | BlogUser        | 统一用户和作者信息   | 继承 AbstractUser，被文章和评论引用    |
| blog          | BaseModel       | 业务模型抽象基类    | 被 Article、Category 和 Tag 继承 |
| blog          | Article         | 文章或独立页面     | 关联用户、分类，并与标签多对多             |
| blog          | Category        | 文章分类树       | 自关联父子分类，与文章一对多              |
| blog          | Tag             | 文章标签        | 与文章多对多                      |
| blog          | Links           | 友情链接        | 独立配置表                       |
| blog          | SideBar         | 侧边栏内容       | 独立配置表                       |
| blog          | BlogSettings    | 网站全局配置      | 单例配置表                       |
| comments      | Comment         | 评论和回复       | 关联用户、文章，自关联父评论              |
| comments      | CommentReaction | 评论 Emoji 反应 | 关联评论和用户                     |
| oauth         | OAuthUser       | 第三方用户映射     | 可绑定 BlogUser                |
| oauth         | OAuthConfig     | OAuth 平台配置  | 独立配置表                       |
| servermanager | commands        | 运维命令说明      | 独立辅助表                       |
| servermanager | EmailSendLog    | 邮件发送日志      | 独立日志表                       |

## 4 数据模型 UML 类图

下图为系统数据模型的整体类图，包含用户、文章、分类、标签、评论、评论反应、OAuth 和辅助配置模型。类图以图片形式嵌入本文档，原始 SVG 和 PlantUML 文件保存在 doc/diagrams 目录。



## 5 数据模型关系分析

### 5.1 用户模型

BlogUser 继承 Django 的 AbstractUser，复用用户名、密码、邮箱、权限和后台登录能力，同时扩展昵称、创建时间、修改时间和来源字段。settings.py 中的 AUTH\_USER\_MODEL 指向 accounts.BlogUser，因此文章、评论和评论反应的用户外键统一关联该模型。

### 5.2 文章、分类与标签

Article 与 BlogUser 是多对一关系，一个用户可以发表多篇文章；Article 与 Category 是多对一关系，一个分类可以包含多篇文章；Article 与 Tag 是多对多关系，Django 会自动创建中间关系表。Category.parent\_category 是指向自身的外键，用于形成树形分类。

### 5.3 评论与互动

Comment 同时关联用户和文章，并通过 parent\_comment 自关联表达评论回复。CommentReaction 记录用户对评论的 Emoji 反应，unique\_together 约束保证同一用户对同一评论的同一反应类型只能出现一次。

### 5.4 OAuth 与辅助模型

OAuthUser 通过可空的 author 外键绑定站内用户，OAuthConfig 保存第三方平台客户端配置。Links 和 SideBar 通过 sequence 控制显示顺序，BlogSettings 通过 clean() 限制只保留一条站点配置，commands 和 EmailSendLog 用于运维管理和邮件发送追踪。

## 6 数据库逻辑与物理结构

### 6.1 核心数据表

| 数据表               | 对应模型            | 关键字段                                  | 约束或说明           |
|-------------------|-----------------|---------------------------------------|-----------------|
| accounts_bloguser | BlogUser        | username、email、nickname、creation_time | 自定义用户表，继承认证字段   |
| blog_article      | Article         | title、body、author、category、status     | title 唯一，多个组合索引 |
| blog_category     | Category        | name、parent_category、slug、index       | name 唯一，自关联分类树  |
| blog_tag          | Tag             | name、slug                             | name 唯一         |
| blog_article_tags | Article-Tag 中间表 | article_id、tag_id                     | 多对多自动关联表        |

|                          |                 |                                    |                           |
|--------------------------|-----------------|------------------------------------|---------------------------|
| comments_comment         | Comment         | body、author、article、parent_comment | article 和 author 外键，自关联回复 |
| comments_commentreaction | CommentReaction | comment、user、reaction_type         | 三者联合唯一                    |
| oauth_oauthuser          | OAuthUser       | author、openid、type、token           | author 可空，用于绑定站内用户        |
| oauth_oauthconfig        | OAuthConfig     | type、appkey、appsecret、callback_url | 同平台只允许一条配置                |

### 6.2 索引与约束

- Article 对 type、status、pub\_time、views、author 和 category 建立组合索引，服务列表页、热门文章和作者归档查询。
- Comment 对 article、parent\_comment、is\_enable 建立组合索引，服务评论列表和审核查询。
- CommentReaction 的评论、用户和反应类型组成联合唯一约束，避免重复点赞。
- Category.name、Tag.name、Links.sequence 等字段使用唯一约束，保证数据一致性和显示顺序稳定。

## 7 Django 架构说明

Django 使用 MTV 模式组织代码。浏览器请求先经过 Middleware，再由 URLConf 分发到 View；View 调用 Model 和 ORM 访问 MySQL，将查询结果放入 context，最后交给 Template 渲染为 HTTP 响应。

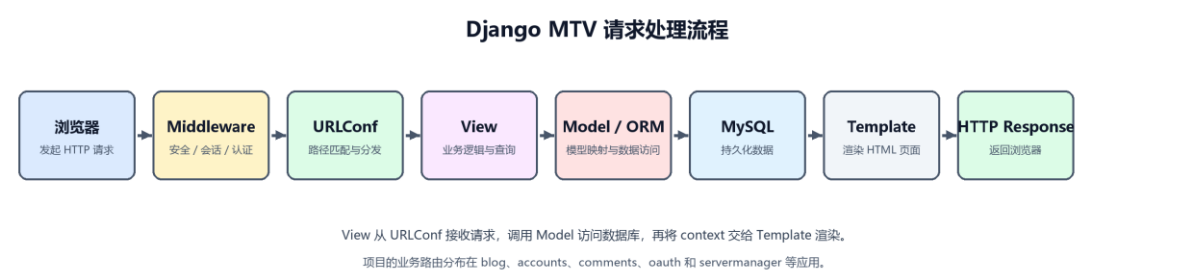


图 7-1 Django MTV 请求处理流程

| 组件         | 作用                    |
|------------|-----------------------|
| URLConf    | 将 URL 路径映射到视图函数或类视图   |
| View       | 接收请求、校验参数、查询模型并返回响应   |
| Model      | 定义数据结构、关系和数据库访问逻辑     |
| Template   | 渲染 HTML，实现页面布局和组件复用   |
| Middleware | 处理安全、会话、认证、语言和缓存等公共逻辑 |

## 8 代码标注说明

| 文件                          | 标注重点                   |
|-----------------------------|------------------------|
| src/accounts/models.py      | 自定义用户模型、继承关系和昵称字段      |
| src/blog/models.py          | 文章、分类、标签、链接、侧边栏和站点配置   |
| src/comments/models.py      | 评论外键、回复自关联和 Emoji 唯一约束 |
| src/oauth/models.py         | 第三方用户绑定和平台配置           |
| src/servermanager/models.py | 运维命令和邮件日志              |
| src/djangoblog/settings.py  | 应用注册、中间件、数据库和认证模型      |
| src/djangoblog/urls.py      | 根路由、错误处理和业务分发          |
| src/blog/views.py           | 视图层职责和 MTV 请求处理流程      |

## 9 结论

DjangoBlog 的数据模型围绕用户、文章和评论三条主线构建。用户模型负责认证和作者身份，文章模型负责内容组织，分类和标签负责内容检索，评论和反应负责用户互动，OAuth 和站点配置负责系统扩展。通过 UML 类图、关系说明和数据库表清单，可以清楚地建立 Django Model、ORM 和 MySQL 数据表之间的对应关系。